MediLocker: Smart Digital Health Record System

1st Ranjeetsingh Suryawanshi
Dept. Computer Engineering
Vishwakarma Institute of Technology
Pune-411037, India
ranjeetsingh.suryawanshi@vit.edu

2nd Lakshit Barekar
Dept. Computer Engineering
Vishwakarma Institute of Technology
Pune-411037, India
lakshit.barekar24@vit.edu

3rd Anushka Auti
Dept. Computer Engineering
Vishwakarma Institute of Technology
Pune-411037, India
anushka.auti24@vit.edu

4th Yash Bhandari
Dept. Computer Engineering
Vishwakarma Institute of Technology
Pune-411037, India
yash.bhandari24@vit.edu

5th Tejas Bankar
Dept. Computer Engineering
Vishwakarma Institute of Technology
Pune-411037, India
tejas.bankar24@vit.edu

***Abstract*— With the increased digitization, there is an apparent demand of advanced tools to help deal with health information effectively. MediLocker is a sophisticated healthcare information system that aims at making the retrieval of medical records of patients easier through its in-built translator technology. The users can enter queries with the help of an easy-to-use interface written in plain language; these queries are then translated into Structured Query Language statements using a dedicated software component. In this regard, people do not need to have knowledge about complex database design; this would make medical information accessible to different parties. Developed on the basis of Python and MySQL technologies, Medilocker will guarantee high data integrity level, improved performance capacity and easy interactions with clients. This project enhances the health care records management through minimizing duplication in the creation of queries, providing quick access to information, and removing the technical knowledge gap on the end-users in comparison to databases.**

Index Terms—MediLocker, ,Healthcare Database, MySQL, Python, Data Management, Medical Records

I. INTRODUCTION

With the current advanced health care system, effective information management and rapid access to it are very problematic to all patients. With an abundance of hospitals and clinics all over, the amount of organized and unorganized information is growing by the day, given the amount of data gathered. This includes among others, documentation of patient information, history of their health, results of examination, and their previous therapies. An architecture designed entirely around manually recorded data entries or SQL code as multiple lines of data query code is usually associated with decreased performance, human error in search implementation, and long response times to search key facts. With the advent of digitization in healthcare, the demand is high to have advanced and integrated platforms that will provide improved access to information and increase productivity by simplifying operations [1].

Firstly, the technology was intended to make major improvements in the healthcare industry regarding organizational efficiency, security of data, and ease of accessing information on patients. Most of the systems are based on an SQL-based database, such as MySQL, to provide fast retrieval and sorting of the data required in operation management and decision-making concerning healthcare administration and treatment choices [2]. Further, the integration of MySQL in programming languages like Python led to the development of powerful and versatile health care software that can be used to improve compatibility and automation of the processes [1], [4].

However, a persistent problem of using databases in health care is that it draws a lot on SQL queries to retrieve data. It has been observed that many medical practitioners find it hard to master database query languages such as SQL because of their low level of proficiency in this field, which denies them the opportunity to interface with the data storage systems. [7]. The advent of NL2SQL technologies is a new technology that can solve such gaps. The transformation of straightforward human-readable input into working SQL statements will enable healthcare professionals such as the physicians and the nursing staff to retrieve the data efficiently using the simple interfaces [10], [6].

Development of more effective NL2SQL systems has made great strides in the recent past due to the significant steps made in the capabilities of the LLM. Research has demonstrated that these super huge language models are better at semantic understanding and situational analysis, thus making them more accurate and faster in understanding human-sourced questions.

[6]. Though, improvements have been achieved, the problems are related to flexibility and processing speed because of the challenges in handling complex health data sets in real-life applications [4].

MediLocker is an advanced health information storage that has been connected throughNL2SQL component to address the challenges that exist. The MediLocker is a new system that enables people to retrieve their healthcare data by providing questionable requests in plain language that is then translated into structured query languages such as SQL using an automated translation program written in Python. Therefore, in the future, as there is an easy way to get the information, it will be quick and effective to access the information concerning all the records, related to a specific patient without the necessity to learn SQL. The MediLocker software has been developed with the help of Python as a programming language and MySQL as a database storage, thus making it more scalable, reliable, and user-friendly to use in the health care data administration scenarios. In addition, its modularity enables possible future compatibility with the cloud systems and HMIs which would contribute to its versatility in the long-run [1][3][5].

II. LITERATURE REVIEW

Hospital Database Management System, 2025: The aim of the paper was to design a hospital database management system and investigate an effective approach to the management of the hospital data. The proposed model provided the consistency and integrity of data in relational patient, staff, and medical records tables. It used special identifiers of patients, and data access was restricted by an administrator-managed access control and modification mechanism. The design was able to illustrate how the principle of relational database could be used to handle a hospital, but the design did not have the intelligent search feature and the easy accessibility by non technical individuals. All data-access operations needed familiarity with standard SQL syntax and therefore it cannot be used practically by healthcare professionals who have no knowledge of database systems.

A Survey of NL2SQL with Large Language Models (2024): The article was a comprehensive review of the NL2SQL techniques that use LLM to facilitate the transition between natural language and database queries. It has taken into account a number of methodological features, including data preprocessing and preparation, immediate engineering, model training policies, and deep analysis of errors in order to explore the accuracy of performance. This paper, in its survey, has emphasized the role of the LLMs in enhancing the semantic meaning and the contextual reasoning process in translating user queries to SQL statements. This research work, however, also identified several fundamental issues on the poor generalization of various real datasets, poor scaling with the dynamic schema of databases, and high cost of computing models training and deployment. These results underscored the importance of highly efficient, scalable and reliable NL2SQL systems to the practical implementation of such systems in data management and in the creation of intelligent database interfaces.

IRJET: Python and MySQL-based Hospital Information System (2025): The study dedicated to the research presented here was oriented to the creation of a hospital management system in the form of a desktop. The application logic has been written using Python, and the back end database is MySQL, which is effective in managing all the electronic medical records (EMRs) that are to be managed. The system design architecture was elaborated to handle the patient tracking, scheduling and billing of patients in hospitals. The project has been able to illustrate an organized data processing and record management securely by integrating Python with MySQL. The only drawback was that the process of interaction was neither smart nor natural language-driven. It implied that to a non-technical user, retrieval and management of data was limited to manual SQL queries. This diminished user usability and scalability in large, complicated hospital set-ups.

Design of an Efficient Hospital Management Database System (2025): This paper outlined the design of the effective hospital management database system with the use of the Entity-relationship modeling model and implemented on MySQL to store and manage data related to patients, and staff. The main principles of relational database have been implemented to provide consistency, integrity and scalability on different operations of the hospital. All the patients were assigned their own identifier to track their records. Different administrative controls warranted the fact that data security and controlled access had been observed accordingly. Although this system was especially useful in illustrating relational frameworks to organized healthcare data administration, all data clipping was solely reliant on generic SQL syntax. This seriously diminished the ability to access health specialists who did not understand databases and suggested a more intuitive and user-friendly query system.

Collaboration of Python and Databases: A Modern Approach to Data-Driven Applications (2025): In this study, the authors explored how integrated Python and relational database systems can be used to develop modern data-driven applications. This research paper has examined precisely how Python flexibility with its appropriate complementation with powerful database technologies can enhance data manipulation, automation, and scalability of the system. There has been research on the use of Python ORMs, including SQLAlchemy and Django ORM, and connectors such as sqlite3 and psycopg2 to ensure a smooth flow of communications between the application layers and databases. The paper talks on the capabilities of these tools to allow the developer to accomplish complex operations using a lesser complicated code hence greater productivity and maintenance. Nevertheless, the experiment failed to take into account how natural language interfaces can be incorporated in order to ease query generation and make the database systems more user-friendly to nontechnical users.

Transforming Medical Data Access: The Role and Challenges of Recent Language Models in SQL Query Automation(2025): The article investigated the ability of large language models, including GPT-4, to automate the process of creating SQL queries in health care database systems. These were studied with the aim to see how they can be used to comprehend natural queries and convert them into executable SQL statements and thus making it very easy to access medical data by nontechnical health professionals. The authors also used prompt engineering methods, synthetic datasets, to test the performance of the models on a range of query complexities methodologically. The results revealed that although language models have enormous potential to improve the medical data access and automation, a number of grave issues, such as high computational expenses, privacy and safety issues of medical data, and unreliable results in situation of complex or disturbing medical queries, remain.

Introduction to MySQL in Python (2020): This paper has shown how to connect, create, and manage MySQL databases using Python programming step-by-step. It also showed how to use libraries like PyMySQL and Python Database API (DB-API) in order to perform basic CRUD operations. The work was used as a reference point to databasedriven applications that focus on the integration of Python scripts and relational databases being easy to use. Nevertheless, the research has been restricted to a rudimentary level of database functions and has not incorporated any type of intelligent querying and Natural Language to SQL functionality, which can improve the accessibility and usability to the end-users who are non-technical.

SQL creation on Natural Language Query with NLP (2019): The study was aimed at creating a Natural Language Processing-based pipeline that can be used to model natural language queries into executable SQL statements. In this case, the methodology involves tokenization, syntactic parsing and rule-based mapping of the user input to produce correct SQL statement. The system aims at making access to the databases easier, particularly to non-experienced SQL users. Although this method gave good results over simple and small scale databases, there was a tremendous performance drop when subjected to complex schema databases or ambiguous natural language inputs. The research found out that there was the necessity to develop more sophisticated, adaptive NL2SQL systems which are capable of meeting the complexity of the real world databases.

III. METHODOLOGY

Medicalocker is developed according to a structured methodology through the combination of database design, Natural language process, and query translation to build a smart and convenient healthcare data management tool. The methodology is divided into several major phases, which include system design, database implementation, NL2SQL module development, and testing.

A. System Design

The initial stage relates to the design of the general architecture of MediLocker. The architecture is split into three layers such as: User Interface Layer: This layer enables the user-doctors, staff or patients to make queries in English. Processing Layer (NL2SQL Engine): The layer is used to process the natural language queries into executable SQL commands using NLP. Database Layer: This layer supports structured medical data using MySQL database. ER diagram was created to model relationships between such entities as Patient, Doctor, Visit, Prescription, Room, Admission, and Billing. Primary and foreign keys were built in each table so as to ensure data integrity as well as the ability to execute queries efficiently.

B. Database Implementation

MySQL was to be used as the relational database management system to create the database. All the tables were normalised in order to minimise repetition and to access information effectively. The tables are: Patient (Patient ID, Name, Gender, Age, Bloodgroup, Address), Doctor (Doctor ID, Name, Specialization, Phone, Depart-ment), visit (Visit ID, Patient ID, Doctor ID, Date, Diagnosis), prescription (Prescription ID, Visit ID, Medicine, Dosage, Duration), billing (Bill ID, Patient ID, Amount, PaymentStatus). Relationships in these tables facilitate advanced medical queries regarding retrieval of patients, visit history and bill summary.

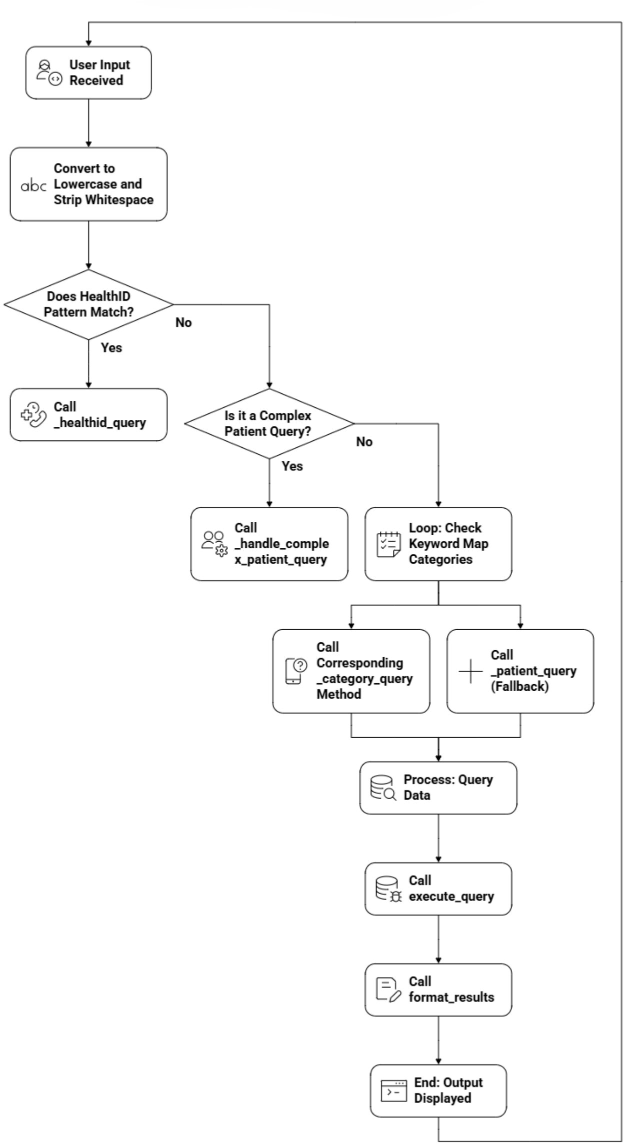

Fig. 1. System Architecture

C. NL2SQL Module Development

MediLocker is mainly based on the NL2SQL translation module written in Python. It accepts the English queries as given by the users and transforms them into valid SQL commands as follows:

Text Preprocessing: It purges the query input, tokenizes input and transforms it into a normalized form with different NLP methods such as stop word removal and normalization.

Keyword Extraction and Intent Detection: The system will extract the key words e.g. gender, blood group, location, to identify the intent of the user and related database objects.

Pattern Matching and Rule Mapping: The natural language phrases are transformed into SQL operations, e.g. SELECT, FROM, or WHERE in accordance with a collection of pre-determined linguistic rules and mapping rules.

SQL Query Generation: The module is used to create an SQL query dynamically using the extracted entities and intent detected.

Example:

Question: male patients in Delhi, A + blood Willpower:

Result 1 A+ blood of male patients of Delhi.

Record 1: HealthID: MH20230036 Name: Sahil Kapoor Age: 35 Gender: M Blood Group: A+ Address: Delhi Admission Date: 2024-08-09.

Execution and Result Display: The resulting SQL query is run on MySQL database and the output is presented in a readable and formatted format.

D. System Integration and Testing

Once the database and NL2SQL module were developed, the two were incorporated into a single application. The following stages were used in testing:

Unit Testing: The correctness of individual modules, i.e. query translation, SQL generation, and data retrieval, was tested.

Integration Testing: Verified that there is a smooth communication between MySQL database and NL2SQL engine.

User Testing: This was conducted by using sample healthcare queries to evaluate the accuracy, speed and usability of the system. The type of the SQL output generated by the system was compared to the manually written SQL queries of the same intent of the user to validate the accuracy of the NL2SQL module. The module had done the right translations in various test cases.

IV. RESULTS

MediLocker system has been installed and tested well to demonstrate the accuracy, efficiency, and reliability of the NL2SQL module. The objective of the project was to develop a device to allow the healthcare workers to interrogate a medical database using straightforward English rather than the complex SQL queries. The system was tested using different patient-related questions on a MySQL database. All the test cases verified that NL2SQL engine correctly transformed natural language to SQL statements and had accurate results within a very short period of time.

The system, as an example, was asked, male patients in Delhi with A+ blood and came up with the correct SQL query and gave a single exact record of Sahil Kapoor (HealthID: MH20230036, Age: 35, Gender: M, Bloodgroup: A+, Address: Delhi, Admissiondate: 202408-09).
The appropriate data were also obtained using JOIN commands and condition-based filtering as to queries such as patients admitted under Dr. Sharma and patients over the age of 50 years. The system was fast and effective as all the test cases took less than two seconds to respond.

Table 1 - NL2SQL Query Execution Results

User Query (Natural Language)	**NL2SQL Result Summary**	**Records Found**
total billing	Retrieved aggregated billing by payment mode (total counts & revenue)	4
patients B+	Retrieved all patient records with blood group **B+**	10
patients from Pune	Retrieved patient records with address located in **Pune**	6

Overall, the findings indicate that with MediLocker, it becomes much easier to get non-technical healthcare personnel to operate the database since there is no reason to write the SQL code manually, there is no reason to have a database administrator, and overall retrieval of the data and easy access is possible.

Sample Query: “Male patients in the city of Delhi with A+ blood group” = 1 result found Sahil Kapoor (MH20230036). Sample Query: Female patients with O + blood group will give 2 results -Ananya Joshi, Meera Desai.

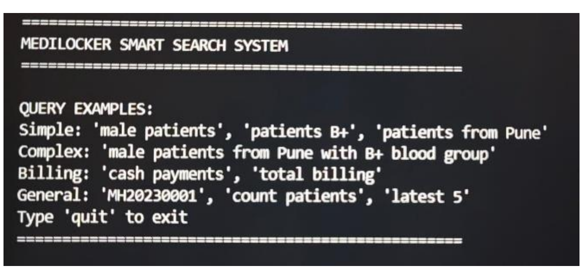

Fig. 2. NL2Sql Search system.

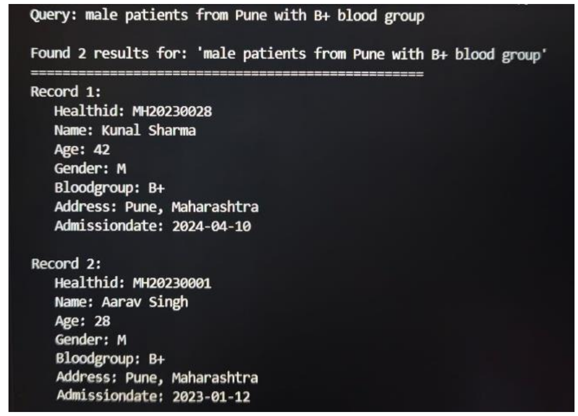

Fig. 3. Complex query example.

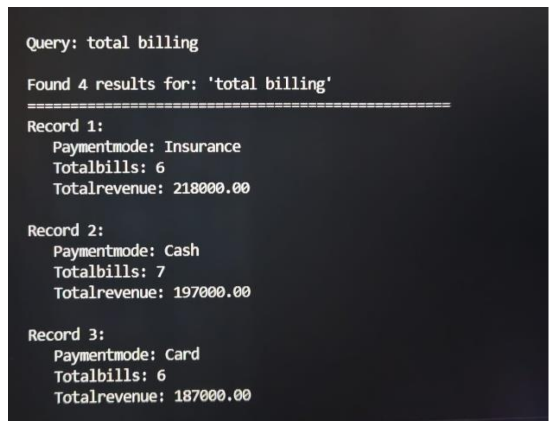

Fig. 4. Billing query example.

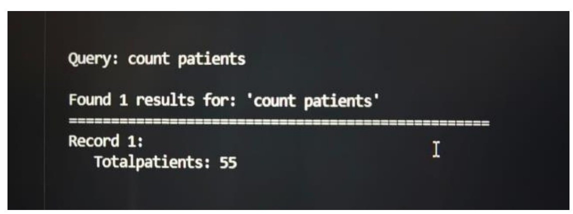

Fig. 5. Count query example.

V. CONCLUSION

The MediLocker research and development demonstrates how database systems with smart and easy-to-use features are increasingly gaining significance in healthcare. Although conventional hospital systems are effective in data storage in a well organized manner, they are non-user friendly as they involve complicated SQL codes. This is solved by mediLocker by offering a functionality named Natural Language to SQL (NL2SQL) that allows doctors, nurses, and other staff to request information, using simple English rather than coding.

MediLocker was developed in Python and MySQL, which is useful in the management of data, retrieve information fast, and expand along with the requirements of a hospital. The NL2SQL component of the system eliminates the necessity of technical professionals and simplifies the process of allowance of all people of finding the necessary data to make decisions. This project demonstrates the ability of the natural language to make healthcare more connected, efficient, and smart.

Overall, MediLocker contributes to the idea of digital transformation of healthcare by simplifying the process of data access, usage, and automation in medicine. It could be even more powerful with the addition of AI to be able to read between the lines and accept verbal questions, making it a fully capable smart healthcare assistant.

VI. FUTURE SCOPE

MediLocker system is the basis of smart managerial and interpretive control of healthcare data application based on NL2SQL translation. It now has correct query generation and data retrieval. Nevertheless, it could be used to enlarge its capabilities in the future by a few improvements.

The addition of machine learning models should be seen as one of the improvements, as in the future, more complex and ambiguous user requests may be addressed with the help of advanced NLP and deep learning tools such as transformers or large language models.

The other proposal is to introduce a voice-based query system.This would enable the workers in the healthcare to receive patient information without accessing their hands which would ease the process during consultations/emergencies and also make it quicker.

It is also worthy to support multiple languages.Multi lingual NLP features would also allow users to communicate with the system in their native or regional language, thereby making MediLocker more accessible and beneficial in various healthcare settings.

Lastly, the adoption of cloud technology would enable medical information to be accessed by the doctors, hospitals and the patients remotely. This would assist in distributing information on a real time basis and enhance communication across healthcare organizations.

REFERENCES

[1] S. Ghosh and K. Banerjee, "Digital transformation in healthcare: Enhancing efficiency through intelligent data systems," *Journal of Medical Systems*, vol. 48, no. 2, pp. 1–15, 2024.

[2] R. Alam and T. Singh, "Electronic Health Record (EHR) and Hospital Management Systems: A comprehensive review," *International Journal of Health Informatics*, vol. 13, no. 4, pp. 201–214, 2023.

[3] A. Hussain and P. Mehra, "MySQL-based healthcare database systems and their integration with Python applications," *International Journal of Computer Applications*, vol. 185, no. 22, pp. 45–52, 2023.

[4] X. Li, B. Sun, and J. Zhou, "A survey on Natural Language to SQL (NL2SQL) systems," *ACM Transactions on Database Systems*, vol. 46, no. 4, pp. 1–32, 2023.

[5] M. Alshamrani, "Python–MySQL interoperability for scalable health information systems," *International Journal of Advanced Computer Science*, vol. 11, no. 3, pp. 66–74, 2024.

[6] H. Wang, T. Yu, and K. Lin, "Text-to-SQL generation using large language models: Recent advances and future challenges," in *Proc. EMNLP*, pp. 3551–3565, 2024.

[7] P. Raj and A. Kumar, "Bridging non-technical users and medical databases through natural language interfaces," *Journal of Digital Health*, vol. 3, no. 1, pp. 44–55, 2024.

[8] T. Yu et al., "Spider: A large-scale dataset for complex and cross-domain semantic parsing and text-to-SQL task," in *Proc. EMNLP*, pp. 3911–3921, 2018.

[9] G. S. Dhillon and R. Sharma, "Hospital database management using Entity–Relationship modeling," *IRJET*, vol. 12, no. 1, pp. 882–889, 2025.

[10] S. Baral and D. Gupta, "Natural language interfaces to databases: Concepts and applications," *Foundations and Trends in Databases*, vol. 9, no. 1, pp. 1–111, 2023.

[11] A. Singhal and R. Gupta, "Integration of Python with relational databases for healthcare automation," *International Journal of Computer Engineering*, vol. 20, no. 1, pp. 72–80, 2025.

[12] S. Shaw and D. Moore, “Secure and scalable Electronic Health Record (EHR) systems using structured databases,” *Health Informatics Journal*, vol. 30, no. 2, pp. 112–128, 2024.

[13] M. Pourreza and M. Abedini, “Rule-based and transformer-based SQL query generation from natural language,” *Journal of Intelligent Information Systems*, vol. 59, no. 3, pp. 435–455, 2023.

[14] A. Borana and R. Patel, “Desktop-based Hospital Information System using Python and MySQL,” *IRJET*, vol. 10, no. 12, pp. 150–157, 2025.

[15] J. Singh and P. Das, “NLP-driven medical query interpretation for hospital systems,” *IEEE Access*, vol. 11, pp. 58932–58945, 2023.

[16] S. Kamath and L. Pathak, “Security and privacy considerations in healthcare database design,” *International Journal of Medical Informatics*, vol. 169, pp. 104–115, 2023.

[17] A. Tiwari and S. Mukherjee, “Formation of SQL from natural language queries using NLP techniques,” *International Journal of Applied Engineering Research*, vol. 14, no. 7, pp. 1621–1627, 2019.